

Compiler Design

Assessment Tool-1 Analytical Problem

Mohammed Fajeeth SA

192511288

CSA14

1) $P = (a * b) + (b * c) * 2$

a) Lexical Analysis (Tokenization)

The lexical analyzer scans the input character stream and groups them into meaningful sequences called tokens. The expression is broken down into 15 distinct tokens

▸ Identifiers: P, a, b, b, c

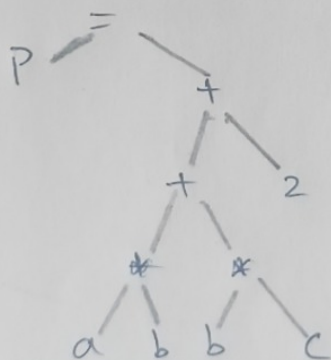
▸ Operators: $=, *, +, *, *$

▸ Separators: $(,), (,)$

▸ Constants: 2

The formal token stream is:

b) Syntax tree



c) Intermediate Code (Three address code)

$$t_1 = a * b$$

$$t_2 = b * c$$

$$t_3 = t_1 + t_2$$

$$t_4 = t_3 + 2$$

$$P_4 = t_4$$

d) Optimized Intermediate Code

No common subexpressions exist

$$t_1 = a * b$$

$$t_2 = b * c$$

$$P = t_1 + t_2 + 2$$

e) Significance of Final Target Code

Target Code is machine/assembly code generated by the compiler

Ex:

```
MOV R1, a
MUL R1, b
MOV R2, b
MUL R2, c
ADD R1, R2
ADD R1, #2
MOV P, R1
```

Importance:

- ↳ Executable by CPU
- ↳ Efficient memory usage
- ↳ Faster execution
- ↳ Final output of compilation process

2) Write five valid strings for each Regular Expression

a) $(aa)^*$

ϵ
aa
aaaa
aaaaaa
aaaaaaaa

d) $(ab)^+$

ab
abab
ababab
abababab
ababababab

b) $(a+tb)^* abb$

abb
aabb
babb
aaabb
bbabb

e) $(aba)^+ (ab)^*$

aba
abaab
abaabab
abaababab
ababaababab

c) $(a+tb) aba (a+tb)$

aba
aaba
abab
abaa
babab

3) Qs Find the number of tokens in the C statement

main()

{ int a=10;

char b='abc';

int c=30;

char d="xyz";

/* comment */

f = m - 40.5;

}

Tokens:

```
main
(  
)  
{  
  int  
  a  
  =  
  10  
  ;  
  char  
  b  
  =  
  'abc'  
  ;  
  int  
  c  
  =  
  30  
  ;  
  char  
  d  
  =  
  "xyz"  
  ;  
  f  
  =  
  m  
  -  
  40.5  
  ;  
}
```

Total tokens = 29

Comment is ignored by lexical analyzer

(ii) Identify lexical errors

```
void main()  
{  
  int x = 10, y = 20;  
  char *;  
  a = &x;  
  x = lab y
```

Errors:

1) char *;

* Identifier missing after *

+ Correct: char *ptr;

2) x = 1ab;

* Invalid Identifier starts with digits

* Correct: x = ab1;

~~3)~~

4) Regular Expressions Alphabet = {0, 1}

(i) Even number of 0's

Regular expression: $1^* (01^*01^*)^*$

(ii) Contains at least one 1

Regular expression = $(0+1)^* 1 (0+1)^*$

(iii) Number of 0's and 1's are odd

Regular expression: $(00+11)^* (01+10)(00+11)^*$

(iv) Starts with 1 and ends with 0

Regular expression: $1(0+1)^*0$

(v) Does not contain consecutive 1's

Regular expression: $(0+10)^* 1 ?$

5)

Lexical Analyzer problem

int var 123 = 12345

float var 456 = 123.45

if (var 123 == var 456) {
return;}

a) How many tokens?

Tokens: int, var 123, =, 12345, ,, float, var 456, =, 123.45, ,, if,
(, var 123, ==, var 456,), {, return, ;, }

Total = 18 tokens

b) List of tokens by category

Keywords = int, float, if, return = 4

Identifiers = var 123, var 456, var 123, var 456 = 4

Operators = =, =, ==, = 3

Separators = ,, ,, (,), {, }, ;, = 7

Integer literal = 12345 = 1

Floating point literal = 123.45 = 1

c) FSM states required

Identifiers FSM: start → Letter → Identifier, states = 2

Integer FSM: start → digit → Integer, states = 2

Float FSM: start → Digit → Digit, states = 4

Operator FSM: states for +, -, *, /, =, ==, <, >, states = 4

Separator FSM: (,), {, }, ;, ,, single recognition states = 2

Total Approximate FSM states = 14 states